```
// 6-Axis Robot Arm - inverse kinematics controller
// Board: Arduino Mega 2560 (6 x TMC2209 / A4988 step-dir drivers)
// Library: AccelStepper (install via Library Manager)
//
// Serial commands (115200 baud):
//   G X Y Z R   -> move tool tip to X,Y,Z (mm) with tool roll R (deg), tool
pointing down
//   J a1 a2 a3 a4 a5 a6 -> move joints directly (deg)
//   H           -> home all joints
//
// Angle conventions (match the simulation):
//   J1 = base yaw, 0 = facing +X
//   J2 = shoulder, 0 = upper arm vertical, + = leaning forward
//   J3 = elbow, 0 = forearm in line with upper arm, + = bending forward
//   J4 = forearm roll, J5 = wrist pitch (0 = straight), J6 = tool roll

#include <AccelStepper.h>
#include <MultiStepper.h>

// ---- Arm geometry (mm) - keep in sync with SolidWorks global variables ----
const float H1 = 110.0;  // base bottom to shoulder axis
const float L2 = 160.0;  // shoulder to elbow
const float L3 = 150.0;  // elbow to wrist centre
const float D6 = 55.0;   // wrist centre to tool tip

// ---- Drive settings ----
const float STEPS_PER_REV = 200.0;
const float MICROSTEPS = 16.0;
const float RATIO[6] = {5.0, 5.0, 4.0, 3.0, 3.0, 1.0};  // gear / belt reductions
const float MIN_DEG[6] = {-170, -90, -135, -180, -110, -180};
const float MAX_DEG[6] = { 170,  90,  135,  180,  110,  180};

// ---- Pins (change to match your wiring) ----
const uint8_t STEP_PIN[6]   = {54, 60, 46, 26, 36, 47};
const uint8_t DIR_PIN[6]    = {55, 61, 48, 28, 34, 49};
const uint8_t ENABLE_PIN[6] = {38, 56, 62, 24, 30, 51};
const uint8_t LIMIT_PIN[6]  = {3, 14, 18, 2, 15, 19};   // switches to GND
const float HOME_DEG[6]     = {-170, -90, -135, -180, -110, -180};  // angle at
each switch

AccelStepper motors[6] = {
  AccelStepper(AccelStepper::DRIVER, 54, 55),
  AccelStepper(AccelStepper::DRIVER, 60, 61),
  AccelStepper(AccelStepper::DRIVER, 46, 48),
```

```
  AccelStepper(AccelStepper::DRIVER, 26, 28),
  AccelStepper(AccelStepper::DRIVER, 36, 34),
  AccelStepper(AccelStepper::DRIVER, 47, 49)
};
MultiStepper group;

long degToSteps(int j, float deg) {
  return lround(deg * STEPS_PER_REV * MICROSTEPS * RATIO[j] / 360.0);
}

// Inverse kinematics: tool tip (x, y, z) in mm, tool pointing straight down.
// Returns false if the point is out of reach.
bool solveIK(float x, float y, float z, float rollDeg, float outDeg[6]) {
  // Wrist centre sits D6 above the tool tip
  float wx = x, wy = y, wz = z + D6;

  float j1 = atan2(wy, wx);
  float r  = sqrt(wx * wx + wy * wy);  // horizontal distance
  float v  = wz - H1;                  // height above shoulder
  float d  = sqrt(r * r + v * v);

  if (d > L2 + L3 - 1.0 || d < fabs(L2 - L3) + 1.0) return false;

  float phi  = atan2(r, v);  // angle of target from vertical
  float beta = acos((L2 * L2 + d * d - L3 * L3) / (2.0 * L2 * d));
  float a    = phi - beta;   // elbow-up solution

  float ex = L2 * sin(a), ey = L2 * cos(a);       // elbow position
  float b  = atan2(r - ex, v - ey);               // forearm angle from vertical

  outDeg[0] = degrees(j1);
  outDeg[1] = degrees(a);
  outDeg[2] = degrees(b - a);
  outDeg[3] = 0.0;
  outDeg[4] = degrees(PI - b);  // keeps tool pointing down
  outDeg[5] = rollDeg - outDeg[0];

  for (int j = 0; j < 6; j++) {
    if (outDeg[j] < MIN_DEG[j] || outDeg[j] > MAX_DEG[j]) return false;
  }
  return true;
}

void moveJoints(const float deg[6]) {
  long target[6];
  for (int j = 0; j < 6; j++) target[j] = degToSteps(j, deg[j]);
```

```
  group.moveTo(target);
  group.runSpeedToPosition();  // blocking, all joints arrive together
}

void homeAll() {
  for (int j = 0; j < 6; j++) {
    motors[j].setSpeed(-400);
    while (digitalRead(LIMIT_PIN[j]) == HIGH) motors[j].runSpeed();
    motors[j].setCurrentPosition(degToSteps(j, HOME_DEG[j]));
  }
  float zero[6] = {0, 0, 0, 0, 0, 0};
  moveJoints(zero);
  Serial.println("Homed");
}

void setup() {
  Serial.begin(115200);
  for (int j = 0; j < 6; j++) {
    pinMode(ENABLE_PIN[j], OUTPUT);
    digitalWrite(ENABLE_PIN[j], LOW);  // enable driver
    pinMode(LIMIT_PIN[j], INPUT_PULLUP);
    motors[j].setMaxSpeed(2000);
    motors[j].setAcceleration(1500);
    group.addStepper(motors[j]);
  }
  Serial.println("Ready. Send H to home first.");
}

void loop() {
  if (!Serial.available()) return;
  char cmd = toupper(Serial.read());

  if (cmd == 'H') {
    homeAll();
  } else if (cmd == 'G') {
    float x = Serial.parseFloat(), y = Serial.parseFloat();
    float z = Serial.parseFloat(), r = Serial.parseFloat();
    float q[6];
    if (solveIK(x, y, z, r, q)) {
      moveJoints(q);
      Serial.println("OK");
    } else {
      Serial.println("Out of reach");
    }
  } else if (cmd == 'J') {
    float q[6];
```

```
    for (int j = 0; j < 6; j++) q[j] = constrain(Serial.parseFloat(), MIN_DEG[j],
MAX_DEG[j]);
    moveJoints(q);
    Serial.println("OK");
  }
}
```